openvfx

Bend a character arm like a film VFX artist, in three sessions.

PAN IIT OSS explainer • secondary school to diploma / undergraduate (levels 3-5)

Why this exists

Every animated character is a surface of thousands of points that must follow an invisible skeleton. This explainer shows the mathematics behind that, using only rotations and matrix products, so you can build it yourself.

The end goal

Bend a character arm for film VFX using matrix transformations and skinning. The end goal is character rigging as used in film VFX and games: a skeleton moves, and a skin that behaves like flesh follows it. The same mathematics drives robot arms and the deformation of real materials.

How the three sessions build up

1. **Session 1: One matrix moves a whole arm** (Level 1, conceptual interactive model, 60 min).
2. **Session 2: Bones rotate about joints, and joints chain** (Level 2, mathematical engineering model, 75 min).
3. **Session 3: Skin that follows the bones** (Level 3, simulation and validation, 90 min).

1 Session 1: One matrix moves a whole arm

A. Real-world problem

Every frame of an animated film moves thousands of points on a character. Nobody moves them one at a time: one matrix moves them all at once. When a character waves, the arm must swing round without stretching or shrinking. How do we write 'turn everything by 30 degrees' so that a computer can do it for every point?

B. Concept and theory

A point is a pair of coordinates (x, y). Rotating it counter-clockwise by an angle theta about the origin gives new coordinates found from the rotation matrix R(theta). Multiplying a matrix by a point means: each new coordinate is a row of the matrix multiplied term by term with (x, y) and added. The two columns of R are unit-length and perpendicular, which is why R keeps lengths and angles; its determinant 1 says that area is kept too.

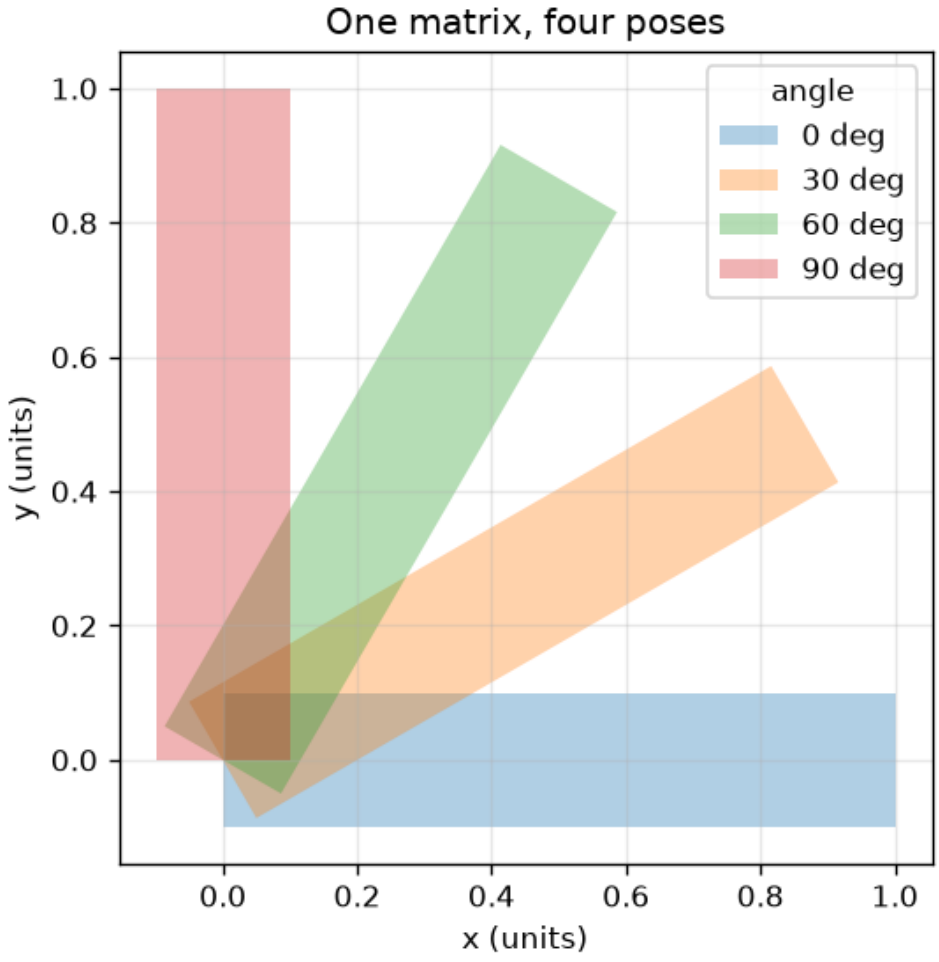
- theta: rotation angle, counter-clockwise (degrees or radians)
- (x, y): coordinates of a point (units (any length unit))
- R: rotation matrix (no unit)
- L: arm length (distance shoulder to hand) (units)

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = R(\theta) \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \end{pmatrix}$$

$R(\theta)^{\mathsf{T}} R(\theta) = I$ (columns are unit length and perpendicular, so lengths are kept)

$$\det R(\theta) = \cos^2\theta + \sin^2\theta = 1 \quad \text{(area is kept)}$$



One matrix, four poses: the arm keeps its length and thickness.

E. Worked example

Where does the hand end up after the arm (length 1, pointing along x) is turned by 30 degrees? Check that the length is still 1.

1. The hand starts at (1, 0). For 30 degrees, cos = 0.866 and sin = 0.500.
2. New x = 0.866 * 1 - 0.500 * 0 = 0.866.
3. New y = 0.500 * 1 + 0.866 * 0 = 0.500.

4. Length = sqrt(0.866^2 + 0.500^2) = sqrt(0.750 + 0.250) = sqrt(1.000) = 1.

Answer. The hand lands at (0.866, 0.500) and the length is still 1.

F. Questions

1. **Conceptual.** Why can a rotation never stretch the arm? What would a matrix that stretched the arm have to look like?
2. **Numerical.** Rotate the point (2, 0) by 90 degrees counter-clockwise. Where does it land? Check its distance from the origin.
3. **Graph and simulation interpretation.** In the graph of hand x and y against angle, at which angle do the two curves cross, and where is the hand then?
4. **Design and optimisation challenge.** An animation needs the hand to swing from pointing right to pointing straight up in exactly 3 equal steps (frames). Choose the angle for each frame and list the hand position in each.

2 Session 2: Bones rotate about joints, and joints chain

A. Real-world problem

When you wave, your hand travels because your forearm rides on your elbow, which rides on your shoulder. A character rig is exactly that chain of joints. Can a cartoon arm with a 30 cm upper arm and a 25 cm forearm touch a button 35 cm across and 20 cm up? And if so, with what joint angles?

B. Concept and theory

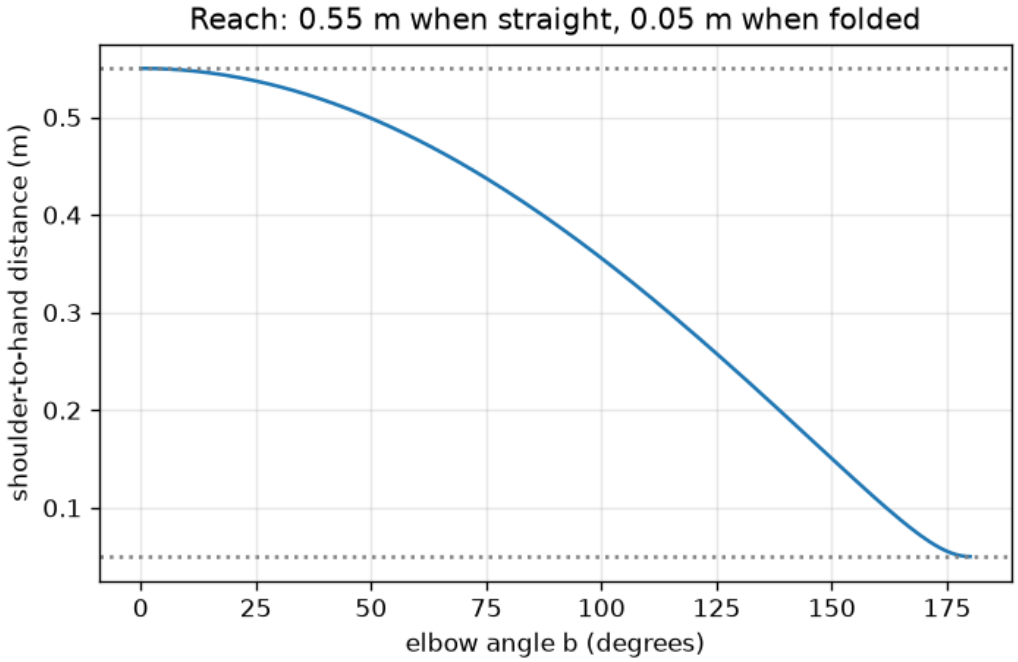
Each bone turns about its own joint. To rotate about a pivot c, move the pivot to the origin, rotate, and move back. The elbow angle b is measured relative to the upper arm, so the forearm points at the sum a + b: angles add down the chain. The distance from shoulder to hand depends only on b (law of cosines).

- a: shoulder angle from the x axis (degrees)
- b: elbow angle relative to the upper arm (degrees)
- L1: upper arm length (0.30) (m)
- L2: forearm length (0.25) (m)
- e, h: elbow and hand positions (x, y) (m)
- c: pivot point of a rotation (m)

$$\mathbf{x}' = R(\theta)(\mathbf{x} - \mathbf{c}) + \mathbf{c}$$

$$\mathbf{e} = L_1(\cos a,\ \sin a), \qquad \mathbf{h} = \mathbf{e} + L_2\big(\cos(a+b),\ \sin(a+b)\big)$$

$$|\mathbf{h}|^2 = L_1^2 + L_2^2 + 2L_1L_2\cos b$$



Reach against elbow angle: 0.55 m when straight, 0.05 m when folded.

E. Worked example

With L1 = 0.30 m, L2 = 0.25 m, a = 40 degrees and b = 50 degrees, find the elbow, the hand and the shoulder-to-hand distance.

1. Elbow = 0.30 * (cos 40, sin 40) = 0.30 * (0.766, 0.643) = (0.230, 0.193) m.
2. The forearm points at a + b = 90 degrees, so its offset is 0.25 * (cos 90, sin 90) = (0, 0.25) m.
3. Hand = elbow + offset = (0.230, 0.443) m.
4. Check with the reach formula: |h|^2 = 0.09 + 0.0625 + 2*0.30*0.25*cos 50 = 0.1525 + 0.0964 = 0.2489, so |h| = 0.499 m.
5. Check with the coordinates: sqrt(0.230^2 + 0.443^2) = 0.499 m. The two methods agree.

Answer. Elbow (0.230, 0.193) m, hand (0.230, 0.443) m, distance 0.499 m.

F. Questions

1. **Conceptual.** Why does the forearm point at angle a + b rather than just b?
2. **Numerical.** For L1 = 0.30 m, L2 = 0.25 m, a = 90 degrees and b = -90 degrees, find the hand position and its distance from the shoulder.
3. **Graph and simulation interpretation.** From the reach graph, what are the largest and smallest shoulder-to-hand distances, and at which elbow angles?
4. **Design and optimisation challenge.** Design: find joint angles that put the hand on (0.35, 0.20) m. How many solutions are there? Is (1.0, 0.0) m reachable?

3 Session 3: Skin that follows the bones

A. Real-world problem

A skeleton has no surface. The skin is a cloud of points and each point must decide which bone to follow. If every point follows exactly one bone, the elbow cracks open like a robot's. If points follow a weighted blend of two bones, the skin looks like flesh, but the elbow gets thinner when it bends. Game engines and film studios use this method (linear blend skinning) and fight this thinning every day. Can you measure it, predict it, and keep it within limits?

B. Concept and theory

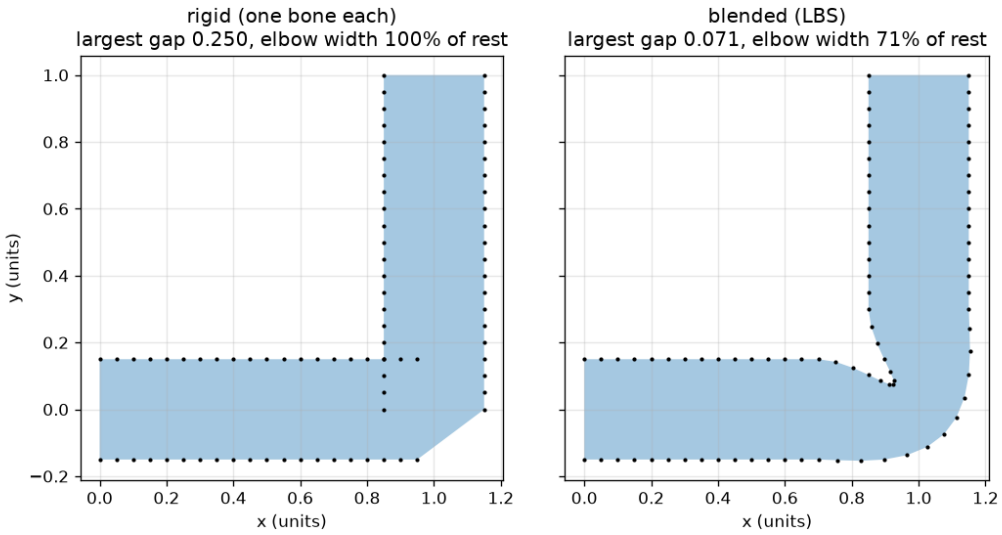
Bone 1 stays put. Bone 2 turns by the bend angle b about the elbow. Each skin point has a weight w between 0 and 1 telling how much it follows bone 2. The new position is the weighted average of where bone 1 and bone 2 would put it. Smooth weights (a smoothstep ramp over a blend zone of half-width z) remove cracks. At the elbow w = 1/2, and averaging a point on the top edge with a point on the bottom edge shrinks their distance to cos(b/2) of the rest width.

- b: elbow bend angle (degrees)
- w: weight: how much a point follows bone 2 (no unit)
- z: half-width of the blend zone (units (arm length 2))
- h: half thickness of the arm (0.15) (units)
- gap: largest distance between neighbouring skin points (units)
- ratio: elbow width divided by rest width (0.30) (no unit)

$$\mathbf{v}' = (1 - w)\,\mathbf{v} + w\,\big(R(b)(\mathbf{v} - \mathbf{c}) + \mathbf{c}\big)$$

$$w(x) = 3t^2 - 2t^3, \qquad t = \mathrm{clip}\left(\frac{x - (1 - z)}{2z},\, 0,\, 1\right)$$

$$\text{elbow width ratio} = \cos\left(\frac{b}{2}\right)$$



A 90 degree bend: one bone per point (cracks) versus blended weights (thinner elbow).

E. Worked example

For a bend of 90 degrees and half thickness h = 0.15, find the blended elbow width and compare it with the hand-calculation cos(b/2).

1. At the elbow x = 1 the weight is w = 1/2. Bone 1 keeps the top point at (1, 0.15); bone 2 turns it about the elbow to (0.85, 0).
2. Blended top point = average = (0.925, 0.075). In the same way the bottom point blends (1, -0.15) and (1.15, 0) into (1.075, -0.075).
3. Width = distance between them = sqrt(0.15^2 + 0.15^2) = 0.212.
4. Rest width = 2h = 0.30, so the ratio is 0.212 / 0.30 = 0.707.
5. Hand calculation: cos(90 / 2) = cos 45 = 0.707. They agree.

Answer. The elbow keeps 70.7 percent of its width; the one-bone skin keeps 100 percent but cracks (largest gap 0.25 against a rest gap of 0.05).

F. Questions

1. **Conceptual.** Why does averaging the positions from two bones make the elbow thinner, while following a single bone does not?
2. **Numerical.** Use the hand calculation to find the elbow width ratio at a bend of 120 degrees. What width is that for an arm 0.30 thick?
3. **Graph and simulation interpretation.** On the width graph, at about which bend does the blue curve cross the red 80 percent line? What does that tell an animator?
4. **Design and optimisation challenge.** Design: choose a blend zone and the largest whole-degree bend so that the elbow keeps at least 80 percent width and the largest gap stays below 0.08. Justify your choice.

4 Real-world mini-project

Rig and animate a waving cartoon arm. A studio hands you a cartoon arm and a four-frame wave: elbow bends of 0, 40, 70 and 40 degrees. The arm must not crack open and must not look thin. Use `lbs_pose` from Session 3 and find the smallest blend zone (in steps of 0.05) for which all four frames pass.

5 The path ahead

1. **Session 4: Helper bones at the elbow.** A three-bone arm (upper arm, helper, forearm) skinned with weights.
2. **Session 5: Paint the weights.** A small weight editor with a sliders-per-region interface that reports gap and width.
3. **Session 6: Blend rotations, not matrices (2D).** A second skinning function with constant width along the arm.
4. **Session 7: Go 3D: twist and quaternions.** The same arm in 3D with a twisting forearm.
5. **Session 8: Animate a wave cycle.** A looping animation exported as a GIF.

6 Answers and hints

Session 1

1. **Conceptual.** Hint: Look at the length of each column of R, and ask what the determinant does and does not tell you. Solution: Each column of R is a vector of length 1 and the two columns are perpendicular (R^T R = I), so R keeps lengths and angles. The determinant cos^2 + sin^2 = 1 only tells us that area is kept: a shear such as [[1, 1], [0, 1]] also has determinant 1 but stretches things (it sends (1, 1) to (2, 1)). A matrix that stretches the arm, such as [[2, 0], [0, 1]] which doubles x, has a column longer than 1.
2. **Numerical.** Hint: For 90 degrees cos = 0 and sin = 1. Apply new x = x cos - y sin and new y = x sin + y cos. Solution: new x = 2*0 - 0*1 = 0 and new y = 2*1 + 0*0 = 2, so the point lands at (0, 2). Distance = sqrt(0^2 + 2^2) = 2, the same as before. Units: same unit as the input.
3. **Graph and simulation interpretation.** Hint: They cross where cos = sin. Solution: They cross at 45 degrees (and again at 225 degrees). At 45 degrees x = y = 0.707, so the hand is at (0.707, 0.707). Reading the graph, both curves have the value 0.707 there.
4. **Design and optimisation challenge.** Hint: The total turn is 90 degrees; divide it into three equal steps and use R for each cumulative angle. Solution: Each step is 30 degrees, so the frames are at 30, 60 and 90 degrees. Hand positions: (0.866, 0.500), (0.500, 0.866) and (0.000, 1.000). Assumption: constant speed. A real animator would also ease in and out.

Session 2

1. **Conceptual.** Hint: Where is the elbow angle b measured from? Solution: b is measured relative to the upper arm, not relative to the x axis. The upper arm already points at a, so the forearm points at a plus whatever extra turn the elbow adds. Angles accumulate down the chain.
2. **Numerical.** Hint: The forearm points at a + b = 0 degrees. Solution: Elbow = 0.30 * (0, 1) = (0, 0.30) m. Forearm offset = 0.25 * (1, 0) = (0.25, 0). Hand = (0.25, 0.30) m. Distance = sqrt(0.0625 + 0.09) = 0.391 m. Check: reach formula gives sqrt(0.1525 + 0.15 cos(-90)) = sqrt(0.1525) = 0.391 m.
3. **Graph and simulation interpretation.** Hint: Read the highest and lowest points of the curve and the matching x values. Solution: The largest is 0.55 m = L1 + L2 at b = 0 degrees (straight arm). The smallest is 0.05 m = |L1 - L2| at b = 180 degrees (forearm folded back). Both are the ends of the curve, where cos b = 1 and -1.
4. **Design and optimisation challenge.** Hint: First find b from |h|^2 = 0.35^2 + 0.20^2 using the reach formula, then find a. Solution: |h|^2 = 0.1625, so cos b = (0.1625 - 0.1525) / 0.15 = 0.0667 and b = +86.2 or -86.2 degrees. For each b, a = atan2(0.20, 0.35) - atan2(0.25 sin b, 0.30 + 0.25 cos b): a = -8.5 degrees for b = +86.2 (elbow down) and a = 68.0 degrees for b = -86.2 (elbow up). Two solutions. (1.0, 0.0) is 1 m away but the longest reach is 0.55 m, so it is unreachable.

Session 3

1. **Conceptual.** Hint: Picture the top and bottom points ending up near each other after the two bones turn them in different directions. Solution: Following one bone moves a top point and its matching bottom point by the same rigid motion, so their distance does not change. Averaging two different motions pulls a top point and a bottom point towards the middle: the average of two rotated points is closer to the centre than either, so the distance between them shrinks. Averaging matrices is not a rotation any more.
2. **Numerical.** Hint: ratio = cos(b/2). Solution: ratio = cos(60 degrees) = 0.5. Width = 0.5

* 0.30 = 0.15 (units). The elbow has lost half its width, much worse than at 90 degrees.

3. **Graph and simulation interpretation.** Hint: Solve cos(b/2) = 0.8. Solution: b/2 = arccos(0.8) = 36.9 degrees, so b = 73.7 degrees. Below about 74 degrees the elbow keeps at least 80 percent of its width; above it, the skin looks visibly pinched. An animator must limit the bend or add helper bones.
4. **Design and optimisation challenge.** Hint: Use lbs_pose(bend, zone) and print the gap and width ratio for a few zones and bends. Solution: Width needs cos(b/2) >= 0.8, so b <= 73.7 degrees: the largest whole-degree bend is 73. At 73 degrees with zone 0.2 the largest gap is 0.075 (below 0.08), so zone 0.2 works; zone 0.15 gives 0.083 at 70 degrees and fails. A safer choice with margin is zone 0.3 and bends up to 70 degrees: width 0.82, gap 0.067. Trade-off: a wider zone reduces the gap but blurs where the elbow is.

7 How to join

Read `CONTRIBUTING.md` in the repository, then pick an issue labelled `good first issue`. Repository: https://github.com/KrunalPanchal1995/openvfx